

PART A

(PART A: TO BE REFERRED BY STUDENTS)

Practical No. 1

A.1 Aim:

1. Explore the concept of software quality assurance.
2. Prepare sample code for carrying out various testing activities.
3. Explore and setup free tools available in different areas of testing

A.2 Prerequisite:

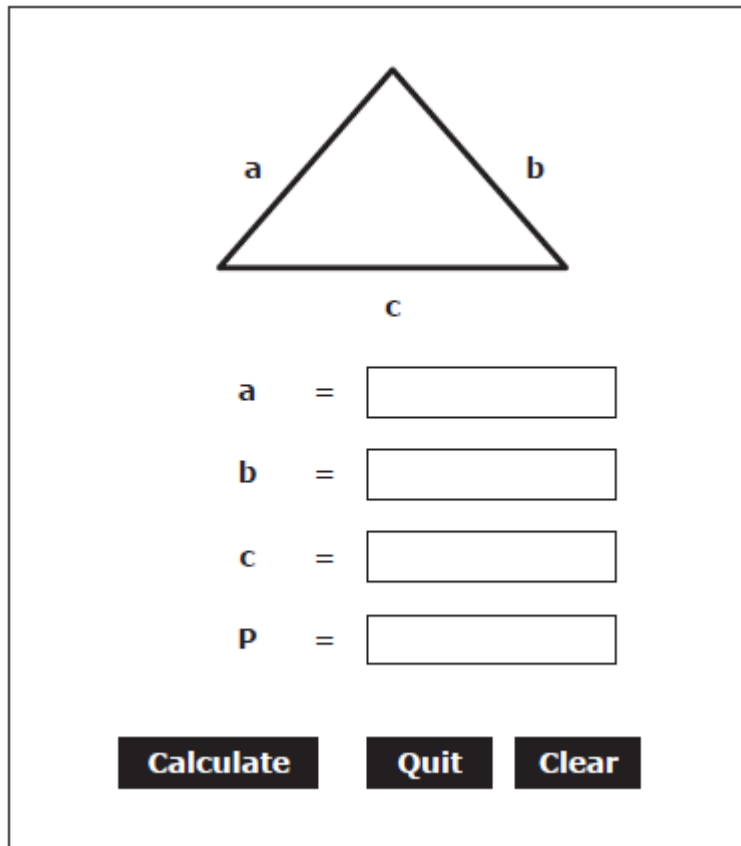
1. PC with Windows operating system.

A.4 Tasks:

1. Write an object-oriented standalone, web-based and mobile program in Python / Java / C++. This application is intended to create a database for employees in an organization. All the fields are mandatory. Records can be added, searched, and deleted as required. [Carried by ODD number group]

Name	
Surname	
Address	
Add	Delete
Search	Quit

2. Write a standalone, web based and mobile program in Python / Java / C++ to compute the perimeter of the triangle. User Interface of which is shown below.
[Carried by EVEN number group]



The image shows a user interface for a triangle perimeter calculator. At the top, there is a diagram of a triangle with its three sides labeled 'a', 'b', and 'c'. Side 'a' is the left side, 'b' is the right side, and 'c' is the bottom side. Below the triangle, there are four input fields, each preceded by a label and an equals sign: 'a =', 'b =', 'c =', and 'p ='. Each label is in a bold, black font. The input fields are empty rectangular boxes. At the bottom of the interface, there are three buttons: 'Calculate', 'Quit', and 'Clear'. The buttons are black with white text.

3. Carry out the setup of anyone freeware / opensource tool in each category mentioned below:
- a. Defect Tracking Tool – Bugzilla / XLS Template
 - b. Regression Testing Tool – **WinRunner** / SilkTest / **Visual Test** / **e-Tester** / **e-Monitor** / jTest / **Jemmy** / **TestGUI** / **JStudio** **SiteWalker** / **XRunner**
 - c. Stress Testing Tool – LoadRunner
 - d. Unit Testing – jTest
 - e. Test Management Tools - **Mercury Test Director** / **QMTTest** / **SMARTS** / **QADirector** / **Test Case Manager**
 - f. Configuration Management Tools: SVN / **Tortoise** / **Microsoft VSS**
 - g. VAPT Testing Tool – Acunetix
4. Understand the different tools
5. Document your findings

A.3 Outcome:

1. Students to prepare themselves for software quality assurance activities

A.4 Theory:

1. <https://www.javatpoint.com/quality-assurance>
2. <http://groups.umd.umich.edu/cis/course.des/cis375/ppt/>
- 3.

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed (chandravadanp@outlook.com) to the concerned lab in charge faculties at the end of the practical in case the there is no Black board access available)

Roll. No. C014	Name: N Mehul
Class B.Tech CE	Batch: 2023-2027
Date of Experiment:	Date of Submission:
Grade:	

B.1 Excel sheet developed by a student:

TASK 1:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;

// Employee Class
class Employee {
    String name, surname, address;

    public Employee(String name, String surname, String address) {
        this.name = name;
        this.surname = surname;
        this.address = address;
    }
}

// Database Class
class EmployeeDatabase {
    ArrayList<Employee> records = new ArrayList<>();

    public void addEmployee(Employee emp) {
        records.add(emp);
    }

    public Employee searchEmployee(String name) {
        for (Employee e : records) {
            if (e.name.equalsIgnoreCase(name)) {
                return e;
            }
        }
        return null;
    }

    public boolean deleteEmployee(String name) {
        for (Employee e : records) {
            if (e.name.equalsIgnoreCase(name)) {
```

```

        records.remove(e);
        return true;
    }
}
return false;
}
}

```

// Main GUI Class

```
public class EmployeeApp extends JFrame {
```

```

    JTextField nameField, surnameField, addressField;
    EmployeeDatabase db = new EmployeeDatabase();

```

```

public EmployeeApp() {
    setTitle("Employee Database");
    setSize(400, 250);
    setLayout(new GridLayout(5, 2));
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

```

// Labels & Fields

```

add(new JLabel("Name"));
nameField = new JTextField();
add(nameField);

```

```

add(new JLabel("Surname"));
surnameField = new JTextField();
add(surnameField);

```

```

add(new JLabel("Address"));
addressField = new JTextField();
add(addressField);

```

// Buttons

```

JButton addBtn = new JButton("Add");
JButton deleteBtn = new JButton("Delete");
JButton searchBtn = new JButton("Search");
JButton quitBtn = new JButton("Quit");

```

```

add(addBtn);
add(deleteBtn);
add(searchBtn);
add(quitBtn);

```

// Button Actions

```

addBtn.addActionListener(e -> addEmployee());
searchBtn.addActionListener(e -> searchEmployee());
deleteBtn.addActionListener(e -> deleteEmployee());
quitBtn.addActionListener(e -> System.exit(0));

```

```

setVisible(true);

```

```
}
```

```

void addEmployee() {
    String name = nameField.getText();

```

```

String surname = surnameField.getText();
String address = addressField.getText();

if (name.isEmpty() || surname.isEmpty() || address.isEmpty()) {
    JOptionPane.showMessageDialog(this, "All fields are mandatory!");
} else {
    db.addEmployee(new Employee(name, surname, address));
    JOptionPane.showMessageDialog(this, "Employee Added!");
    clearFields();
}
}

void searchEmployee() {
    String name = nameField.getText();
    Employee emp = db.searchEmployee(name);

    if (emp != null) {
        surnameField.setText(emp.surname);
        addressField.setText(emp.address);
    } else {
        JOptionPane.showMessageDialog(this, "Employee not found!");
    }
}

void deleteEmployee() {
    String name = nameField.getText();

    if (db.deleteEmployee(name)) {
        JOptionPane.showMessageDialog(this, "Employee Deleted!");
        clearFields();
    } else {
        JOptionPane.showMessageDialog(this, "Employee not found!");
    }
}

void clearFields() {
    nameField.setText("");
    surnameField.setText("");
    addressField.setText("");
}

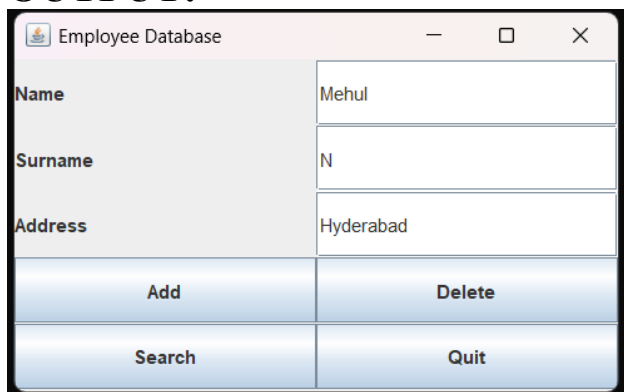
public static void main(String[] args) {
    new EmployeeApp();
}
}

```

Errors:

```
EmployeeApp.java:67: error: cannot find symbol
    address = new JTextField();
    ^
  symbol:   variable address
  location: class EmployeeApp
EmployeeApp.java:79: error: cannot find symbol
    add(quitBtn);
    ^
  symbol:   variable quitBtn
  location: class EmployeeApp
EmployeeApp.java:85: error: cannot find symbol
    quitBtn.addActionListener(e -> System.exit(0));
    ^
  symbol:   variable quitBtn
  location: class EmployeeApp
EmployeeApp.java:93: error: cannot find symbol
    String = addressField.getText();
    ^
  symbol:   variable String
  location: class EmployeeApp
EmployeeApp.java:95: error: cannot find symbol
    if (name.isEmpty() || surname.isEmpty() || address.isEmpty()) {
                                                ^
  symbol:   variable address
  location: class EmployeeApp
EmployeeApp.java:98: error: cannot find symbol
    db.addEmployee(new Employee(name, surname, address));
                                                ^
  symbol:   variable address
  location: class EmployeeApp
6 errors
```

OUTPUT:



Name	Mehul
Surname	N
Address	Hyderabad
Add	Delete
Search	Quit

Task2:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class TrianglePerimeter extends JFrame {

    JTextField aField, bField, cField, pField;

    public TrianglePerimeter() {
        setTitle("Triangle Perimeter");
        setSize(400, 350);
```

```
setLayout(null);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
// Labels
```

```
JLabel aLabel = new JLabel("a =");
aLabel.setBounds(50, 120, 50, 25);
add(aLabel);
```

```
JLabel bLabel = new JLabel("b =");
bLabel.setBounds(50, 160, 50, 25);
add(bLabel);
```

```
JLabel cLabel = new JLabel("c =");
cLabel.setBounds(50, 200, 50, 25);
add(cLabel);
```

```
JLabel pLabel = new JLabel("P =");
pLabel.setBounds(50, 240, 50, 25);
add(pLabel);
```

```
// Text Fields
```

```
aField = new JTextField();
aField.setBounds(100, 120, 200, 25);
add(aField);
```

```
bField = new JTextField();
bField.setBounds(100, 160, 200, 25);
add(bField);
```

```
cField = new JTextField();
cField.setBounds(100, 200, 200, 25);
add(cField);
```

```
pField = new JTextField();
pField.setBounds(100, 240, 200, 25);
pField.setEditable(false);
add(pField);
```

```
// Buttons
```

```
JButton calcBtn = new JButton("Calculate");
calcBtn.setBounds(50, 280, 100, 30);
add(calcBtn);
```

```
JButton clearBtn = new JButton("Clear");
clearBtn.setBounds(160, 280, 80, 30);
add(clearBtn);
```

```
JButton quitBtn = new JButton("Quit");
quitBtn.setBounds(260, 280, 80, 30);
add(quitBtn);
```

```
// Button Actions
```

```
calcBtn.addActionListener(e -> calculatePerimeter());
clearBtn.addActionListener(e -> clearFields());
quitBtn.addActionListener(e -> System.exit(0));
```



```

    setVisible(true);
}

void calculatePerimeter() {
    try {
        double a = Double.parseDouble(aField.getText());
        double b = Double.parseDouble(bField.getText());
        double c = Double.parseDouble(cField.getText());

        if (a <= 0 || b <= 0 || c <= 0) {
            JOptionPane.showMessageDialog(this, "Enter positive values only!");
            return;
        }

        double p = a + b + c;
        pField.setText(String.valueOf(p));

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Please enter valid numbers!");
    }
}

void clearFields() {
    aField.setText("");
    bField.setText("");
    cField.setText("");
    pField.setText("");
}

public static void main(String[] args) {
    new TrianglePerimeter();
}
}

```

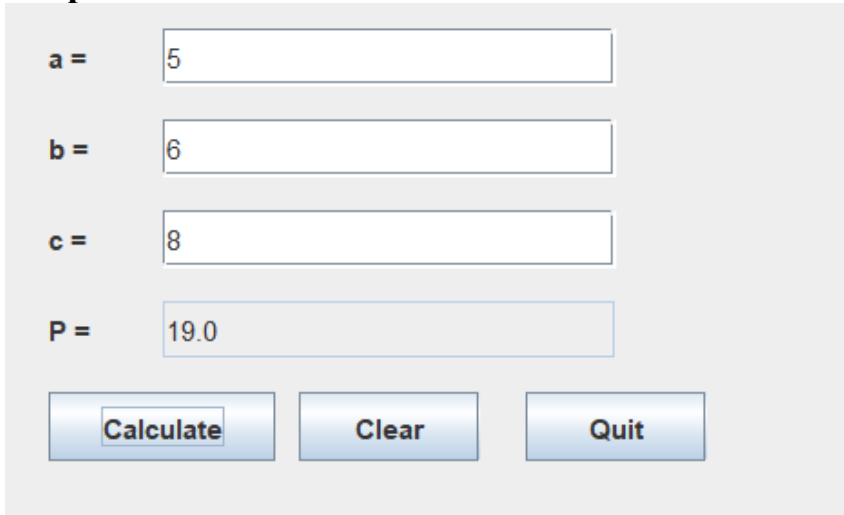
Errors:

```

TrianglePerimeter.java:12: error: cannot find symbol
    set(null);
    ^
symbol:   method set(<null>)
location: class TrianglePerimeter
TrianglePerimeter.java:17: error: cannot find symbol
    aLabel.set(50, 120, 50, 25);
           ^
symbol:   method set(int,int,int,int)
location: variable aLabel of type JLabel
TrianglePerimeter.java:25: error: cannot find symbol
    cLabel.set(50, 200, 50, 25);
           ^
symbol:   method set(int,int,int,int)
location: variable cLabel of type JLabel
TrianglePerimeter.java:86: error: cannot find symbol
    JOptionPane.showMessage(this, "Please enter valid numbers!");
                        ^
symbol:   method showMessage(TrianglePerimeter,String)
location: class JOptionPane
4 errors
error: compilation failed

```

Output:



The screenshot shows a Java Swing window with a light gray background. It contains four text input fields with labels 'a =', 'b =', 'c =', and 'P ='. The values entered are 5, 6, 8, and 19.0 respectively. Below the input fields are three buttons: 'Calculate', 'Clear', and 'Quit'.

B.3 Observations and learning:

During the development of the triangle perimeter program, we learned how to design a simple user interface using Java Swing. We understood how to take user input through text fields and process it using event handling. The program helped us learn how to validate inputs to avoid errors such as empty or non-numeric values. We also learned how to perform basic mathematical calculations in Java and display the results on the screen. This project improved our understanding of object-oriented programming concepts like classes, methods, and encapsulation. Overall, it helped us gain practical experience in building a standalone desktop application.

B.4 Conclusion:

The triangle perimeter application successfully calculates the perimeter using the formula $P = a + b + c$.

It provides a simple and user-friendly interface where users can enter the values of the sides and get instant results. The program includes features like Calculate, Clear, and Quit, making it easy to use. This project demonstrates the practical use of Java and GUI programming for solving real-world mathematical problems. It also shows how basic programming concepts can be applied to create useful applications.